

Netaji Subhas University Of Technology



ECE Department

Electronic Design Workshop Project 2023-24

under the specialized guidance of Mr. D.V.Gadre

Names of Students:

ATUL PAL 2022UEC2621

AYUSHMAN 2022UEC2609

HIMANSHU 2022UEC2611

Abstract

The "The RETRO Snake Game" project presents the development of a classic Snake game using Arduino microcontroller, various components including a buzzer, push buttons, and an alphanumeric LCD display, along with breadboard and PCB board for circuit prototyping and implementation. The Snake game is a popular arcade-style game where the player controls a snake that moves around a bounded area, consuming food pellets, and growing longer. The primary objective of this project is to provide a tangible demonstration of interfacing hardware components with Arduino to create an interactive gaming experience.

The game utilizes push buttons as input devices for controlling the movement of the snake, enabling players to navigate the snake in four directions: up, down, left, and right. A buzzer is employed to provide audio feedback, such as indicating successful consumption of food pellets or collisions with obstacles. Additionally, an alphanumeric LCD display is used to render the game environment, including the snake, food pellets, and score.

The development process involves designing the game logic, implementing the hardware interface with Arduino, and integrating the various components to ensure seamless functionality. The breadboard serves as a platform for initial circuit prototyping, allowing for rapid iteration and testing of different configurations. Once the design is finalized, a PCB board is used to create a more permanent and compact circuit layout, suitable for deployment in the final product.

Acknowledgments

We would like to express our deep gratitude to our project guide and lecturer, Mr. D V Gadre, Department of Electronics and Communication Engineering for his guidance with unsurpassed knowledge and immense encouragement. We are grateful to Prof. D.V. Gadre, Department of Electronics and Communication Engineering, for providing us with the required assets for the completion of the project work.

We express our thanks to all teaching faculty of Department of ECE, whose suggestions during reviews helped us in accomplishment of our project. We would like to thank all non- teaching staff of the Department of ECE, NSUT for providing great assistance in accomplishment of our project. We would like to thank our parents, friends, and classmates for their encouragement throughout our project period. At last but not the least, we thank everyone for supporting us directly or indirectly in completing this project successfully.

Contents

1	Introduction	4
1.1	Ideology	4
1.2	Core	4
2	Components Brought to Use	4
2.1	Arduino Uno	4
2.2	Lcd display	4
2.3	Push buttons	4
2.4	Resistors	4
3	Code	4
4	Result	4
5	Conclusion	4

1 Introduction

1.1 Ideology

1.2 Core

2 Components Brought to Use

2.1 Arduino Uno

2.2 Lcd display

2.3 Push buttons

2.4 Resistors

3 Code

4 Result

5 Conclusion

Introduction

Remember the days when the iconic Snake game was the pinnacle of mobile gaming? Welcome to a journey back in time, where pixels were king, and simple yet addictive gameplay ruled the screens. Now, let's bring back that nostalgic charm with a retro twist by creating our own Arduino-based Snake game! Today, we're diving headfirst into the realm of retro gaming, reviving a beloved classic with a modern twist. Imagine a world where nostalgia intertwines with innovation, where the simplicity of retro gaming meets the sophistication of modern technology. That's exactly what we aim to achieve with our Arduino Snake game project. As we embark on this adventure, we'll be guided by the spirit of exploration and the joy of creation.

But our journey doesn't end there. We'll be joined by an ensemble cast of components, each playing a crucial role in the grand performance that is our Snake game. From the humble buzz of a buzzer to the elegant glow of an LCD display, every element will contribute its own unique flavor to the experience, transforming a simple game into an immersive adventure.

In this project, we'll infuse the classic Snake gameplay with modern hardware components like a buzzer, LCD display, and a custom PCB for a sleek and compact design. Picture this: the unmistakable sound of pixels crunching beneath the sleek surface of an LCD display, the rhythmic melody of a buzzer echoing through the air, and the thrill of guiding a pixelated serpent through a maze of obstacles.

Get ready to experience the joy, the challenge, and the sheer delight of playing a retro Snake game, all powered by the magic of Arduino. So grab your controller, buckle up for a trip down memory lane, and let's make some gaming history together! Get ready to embark on a journey of coding, circuitry, and retro gaming fun!

As we embark on this project together, let's embrace the spirit of curiosity and creativity that defines the world of Arduino. Let's challenge ourselves to push the boundaries of what's possible, to explore new ideas, and to delight in the joy of discovery. Together, we'll create something truly special—a Snake game that not only captures the essence of the past but also paves the way for the future of gaming. So let's roll up our sleeves, fire up our imaginations, and let the adventure begin!

Component Used

Here is a more detailed description of the first three materials used in the project:

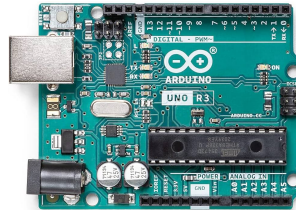
1. Arduino Uno

The Arduino UNO R3 board serves as the cornerstone of our Snake game project, providing the essential hardware platform on which our game will come to life. Its utility in this endeavor is multifaceted and indispensable.

The Arduino board boasts a capable microcontroller that executes the game logic with precision and efficiency. From controlling the snake's movements to managing game mechanics, the Arduino's processing power ensures smooth and responsive gameplay.

Arduino's General Purpose Input/Output (GPIO) pins serve as the interface between the microcontroller and external components. These pins allow us to connect various peripherals essential for the game, including push buttons for directional control, an LCD display for visual feedback, and a buzzer for sound effects.

Arduino's flexibility enables us to tailor the Snake game to our specific requirements. Whether it's customizing gameplay mechanics, adding new features, or integrating additional hardware, the Arduino board adapts effortlessly to our needs, making it the perfect platform for experimentation and innovation.



With its user-friendly programming environment and extensive library support, Arduino simplifies the development process, allowing us to focus on designing a captivating gaming experience without getting bogged down by complex code.

Arduino Uno facilitates rapid prototyping and iteration during the development process. Its ease of use and compatibility with a wide range of sensors, actuators, and displays allow developers to quickly test different gameplay mechanics, iterate on designs, and refine the gaming experience until it meets their vision.

2. **LCD display** The LCD (Liquid Crystal Display) display plays a crucial role in enhancing the user experience and providing visual feedback in a Snake game implemented on Arduino. The LCD display serves as the canvas on which the game grid is visualized. Each pixel or character on the display represents a cell in the game grid, allowing players to see the snake, food, and obstacles as they navigate through the game world.

As the snake moves across the game grid, its position and direction are updated on the LCD display in real-time. This visual feedback enables players to track the movement of the snake and plan their next moves accordingly. The LCD display shows the location of food items within the game grid. When the snake consumes food, the corresponding cell on the display is updated to reflect the change.

Each time the snake eats food, the score is incremented, and the updated score is displayed on the screen. This provides players with feedback on their progress and performance in the game.



When the game ends, either due to the snake colliding with itself or with an obstacle, the LCD display can show a "Game Over" message to notify the player of the end of the game. This allows players to understand the outcome of their gameplay session and encourages them to try again.

The LCD display in a Snake game implemented on Arduino serves as a vital tool for visualizing the game grid, displaying the snake, food, and obstacles, showing the player's score, and communicating important messages such as "Game Over." Its utility lies in providing clear and intuitive visual feedback that enhances the overall gaming experience and immerses players in the world of the game.

3. **Push buttons** Push buttons are essential components in a Snake game implemented on Arduino, serving as the primary method for player input and control. Push buttons are typically assigned to control the movement direction of the snake. For example, one button may be designated for moving the snake up, another for moving it down, one for left, and another for right. When a button is pressed, it signals to the game logic to change the direction of the snake accordingly.



Push buttons provide real-time input feedback, allowing players to control the snake's movement with precision and responsiveness. Each time a button is pressed, the snake's direction is updated immediately, resulting in smooth and fluid gameplay. Push buttons facilitate player interaction with the game environment, enabling them to navigate the snake through the game grid and collect food items while avoiding obstacles and collisions. The tactile feedback of pressing buttons adds to the immersive gaming experience and enhances player engagement.

By offering physical buttons for directional control, push buttons enhance the overall user experience of playing the Snake game. They provide a familiar and intuitive interface that players can easily grasp, regardless of their level of gaming experience. Customization and Adaptability: Push buttons can be customized and adapted to suit the preferences of individual players. For example, players may choose to use different buttons for directional control or adjust the button layout to better accommodate their playing style.

4. **Buzzer** The buzzer in an Arduino Snake game serves multiple functions, enhancing the overall gaming experience. The buzzer generates sound effects that add an auditory dimension to the game. For example, when the snake eats food, collides with obstacles, or achieves certain milestones (such as reaching a high score), the buzzer can produce distinct sounds to provide immediate feedback to the player. The buzzer provides feedback to the player for various in-game actions. Different sounds can indicate different events, such as successful food consumption, collisions, or game over scenarios. This feedback helps players understand their progress and performance within the game.



By adding sound effects, the buzzer enhances the immersion of players in the game environment. The auditory cues created by the buzzer create a more engaging and immersive experience, drawing players deeper into the gameplay and heightening their sense of involvement.

The buzzer can be used to alert players to important events or warn them of impending dangers. For example, it can emit a warning sound when the snake approaches an obstacle or when the game is about to end due to a collision, giving players a chance to react and adjust their strategy accordingly.

The buzzer can be used to alert players to important events or warn them of impending dangers. For example, it can emit a warning sound when the snake approaches an obstacle or when the game is about to end due to a collision, giving players a chance to react and adjust their strategy accordingly.

Overall, the buzzer in an Arduino Snake game serves as a versatile tool for adding sound effects, providing feedback to players, enhancing immersion, delivering alerts and warnings, and enabling customization. Its utility extends beyond mere audio output, enriching the gaming experience and contributing to a more dynamic and engaging gameplay environment.

5. **Resistors** In an Arduino Snake game, resistors play several important roles, contributing to the functionality and reliability of the overall circuitry.

Button Pull-Up Resistors: Resistors are commonly used as pull-up resistors in conjunction with push buttons. When push buttons are not pressed, they are in a floating state, which can lead to unpredictable behavior in the circuit.

Pull-up resistors ensure that the input pin connected to the button reads a stable logic level (usually HIGH) when the button is not pressed, preventing false triggering and ensuring reliable button detection.



Pull-Down Resistors: While less common in Arduino Snake games, pull-down resistors may be used in certain configurations to ensure stable logic levels when push buttons are not pressed.

Pull-down resistors pull the input pin to a LOW logic level when the button is not pressed, serving a similar purpose to pull-up resistors but with the opposite logic polarity.

Overall, resistors in an Arduino Snake game contribute to stable button detection, debounce noisy signals, limit LED current, and ensure the overall reliability and safety of the circuitry. They play a crucial role in maintaining proper functionality and preventing damage to components, making them essential components in the design of the game.

Code

```
#include <Arduino.h>
#include "pitches.h"

enum DisplayItem {GRAPHIC_ITEM_NONE, GRAPHIC_ITEM_A, GRAPHIC_ITEM_B,
GRAPHIC_ITEM_NUM};

#include <LiquidCrystal_I2C.h> //Install LiquidCrystal_I2C.h

#include <stdlib.h> //Install stdlib.h

#include <limits.h> //Install limits.h

#define GRAPHIC_WIDTH 20

#define GRAPHIC_HEIGHT 8

LiquidCrystal_I2C lcd(0x27,20,4);
//buzzer start
#define BUZZER_PIN 9

int melody[] = {
    NOTE_B4, NOTE_B5, NOTE_FS5, NOTE_DS5,
    NOTE_B5, NOTE_FS5, NOTE_DS5, NOTE_C5,
    NOTE_C6, NOTE_G6, NOTE_E6, NOTE_C6, NOTE_G6, NOTE_E6,

    NOTE_B4, NOTE_B5, NOTE_FS5, NOTE_DS5, NOTE_B5,
    NOTE_FS5, NOTE_DS5, NOTE_DS5, NOTE_E5, NOTE_F5,
    NOTE_F5, NOTE_FS5, NOTE_G5, NOTE_G5, NOTE_GS5, NOTE_A5, NOTE_B5
};

int durations[] = {
    16, 16, 16, 16,
    32, 16, 8, 16,
    16, 16, 16, 32, 16, 8,

    16, 16, 16, 16, 32,
    16, 8, 32, 32, 32,
    32, 32, 32, 32, 32, 16, 8
};

//buzzer ends
```

```

byte block[3] = {

B01110,

B01110,

B01110,

};

byte apple[3] = {

B00100,

B01010,

B00100,

};

#define DEBOUNCE_DURATION 20

//Return true if the actual output value is true
bool debounce_activate(unsigned long* debounceStart)
{
    if(*debounceStart == 0)
        *debounceStart = millis();
    else if(millis()-*debounceStart>DEBOUNCE_DURATION)
        return true;
    return false;
}

//Return true if it's rising edge/falling edge
bool debounce_activate_edge(unsigned long* debounceStart)
{

```

```

if(*debounceStart == ULONG_MAX){

return false;

}else if(*debounceStart == 0){

*debounceStart = millis();

}else if(millis()-*debounceStart>DEBOUNCE_DURATION){

*debounceStart = ULONG_MAX;

return true;

}

return false;

}

void debounce_deactivate(unsigned long* debounceStart){

*debounceStart = 0;

}

#define BUTTON_LEFT 6 //LEFT BUTTON

#define BUTTON_RIGHT 7 // RIGHT BUTTON

unsigned long debounceCounterButtonLeft = 0;

unsigned long debounceCounterButtonRight = 0;

struct Pos {

uint8_t x=0, y=0;

};

struct Pos snakePosHistory[GRAPHIC_HEIGHT*GRAPHIC_WIDTH];

//first element is the head.

```

```

size_t snakeLength = 0;

enum {SNAKE_LEFT,SNAKE_UP,SNAKE_RIGHT,SNAKE_DOWN} snakeDirection;

struct Pos applePos;

unsigned long lastGameUpdateTick = 0;

unsigned long gameUpdateInterval = 500;

bool thisFrameControlUpdated = false;

enum {GAME_MENU, GAME_PLAY, GAME_LOSE, GAME_WIN} gameState;

uint8_t graphicRam[GRAPHIC_WIDTH*2/8][GRAPHIC_HEIGHT];

void graphic_generate_characters()

{

/*

space: 0 0

0: 0 A

1: 0 B

2: A 0

3: A A

4: A B

5: B 0

6: B A

7: B B

*/

for (size_t i=0; i<8; i++) {

```

```

byte glyph[8];

int upperIcon = (i+1)/3;

int lowerIcon = (i+1)%3;

memset(glyph, 0, sizeof(glyph));

if(upperIcon==1)

memcpy(&glyph[0], &block[0], 3);

else if(upperIcon==2)

memcpy(&glyph[0], &apple[0], 3);

if(lowerIcon==1)

memcpy(&glyph[4], &block[0], 3);

else if(lowerIcon==2)

memcpy(&glyph[4], &apple[0], 3);

lcd.createChar(i, glyph);

}

delay(1);

//Wait for the CGRAM to be written

}

void graphic_clear(){

memset(graphicRam, 0, sizeof(graphicRam));

}

void graphic_add_item(uint8_t x, uint8_t y, enum DisplayItem item) {

graphicRam[x/(8/2)][y] |= (uint8_t)item*(1<<((x%(8/2))*2));

}

```



```

void graphic_flush(){

lcd.clear();

for(size_t x=0; x<20; x++){

for(size_t y=0; y<4; y++){

enum DisplayItem upperItem =

(DisplayItem)((graphicRam[x/(8/2)][y*2]>>((x%(8/2))*2))&0x3);

enum DisplayItem lowerItem =

(DisplayItem)((graphicRam[x/(8/2)][y*2+1]>>((x%(8/2))*2))&0x3);

if(upperItem>=GRAPHIC_ITEM_NUM)

upperItem = GRAPHIC_ITEM_B;


if(lowerItem>=GRAPHIC_ITEM_NUM)

lowerItem = GRAPHIC_ITEM_B;

lcd.setCursor(x, y);

if(upperItem==0 && lowerItem==0)

lcd.write(' ');

else

lcd.write(byte((uint8_t)upperItem*3+(uint8_t)lowerItem-1));

}

}

}

void game_new_apple_pos()

{

```

```

bool validApple = true;

do{

applePos.x = rand()%GRAPHIC_WIDTH;

applePos.y = rand()%GRAPHIC_HEIGHT;

validApple = true;

for(size_t i=0; i<snakeLength; i++)

{

if(applePos.x==snakePosHistory[i].x && applePos.y==snakePosHistory[i].y){

validApple = false;
//buzzer

break;

}

}

} while(!validApple);

}

void game_init(){

srand(micros());

gameUpdateInterval = 500;

gameState = GAME_PLAY;

snakePosHistory[0].x=3; snakePosHistory[0].y=1;

snakePosHistory[1].x=2; snakePosHistory[1].y=1;

snakePosHistory[2].x=1; snakePosHistory[2].y=1;

```

```

snakePosHistory[3].x=0; snakePosHistory[3].y=1;

snakeLength = 4;

snakeDirection = SNAKE_RIGHT;

game_new_apple_pos();

thisFrameControlUpdated = false;

}

void game_calculate_logic() {

    if(gameState!=GAME_PLAY)

        return;

    //Calculate the movement of the tail

    for(size_t i=snakeLength; i>=1; i--){

        snakePosHistory[i] = snakePosHistory[i-1];

    }

    //Calculate the head movement

    snakePosHistory[0]=snakePosHistory[1];

    switch(snakeDirection){

        case SNAKE_LEFT: snakePosHistory[0].x--; break;

        case SNAKE_UP: snakePosHistory[0].y--; break;

        case SNAKE_RIGHT: snakePosHistory[0].x++; break;

        case SNAKE_DOWN: snakePosHistory[0].y++; break;

    }

    //Look for wall collision

```

```

if(snakePosHistory[0].x<0||snakePosHistory[0].x>=GRAPHIC_WIDTH||snakePosHistory[0].y
<0||snakePosHistory[0].y>=GRAPHIC_HEIGHT){

gameState = GAME_LOSE;

return;

}

//Look for self collision

for(size_t i=1; i<snakeLength; i++){

if(snakePosHistory[0].x==snakePosHistory[i].x &&
snakePosHistory[0].y==snakePosHistory[i].y){

gameState = GAME_LOSE;

return;

}

}

if(snakePosHistory[0].x==applePos.x && snakePosHistory[0].y==applePos.y){
tone(BUZZER_PIN, 1000); //Send 1KHz sound signal
delay(100);
noTone(BUZZER_PIN); //Stop sound

snakeLength++;

gameUpdateInterval = gameUpdateInterval*9/10;

if(snakeLength>=sizeof(snakePosHistory)/sizeof(*snakePosHistory))

gameState = GAME_WIN;

else

game_new_apple_pos();

}

}

```

```

void game_calculate_display() {

    graphic_clear();

    switch(gameState){

    case GAME_LOSE:

        lcd.clear();
        //buzzer
        tone(BUZZER_PIN,440,200); //Send 1KHz sound signal
        delay(200);
        tone(BUZZER_PIN,415,200); //Send 1KHz sound signal
        delay(200);
        tone(BUZZER_PIN,392,200 ); //Send 1KHz sound signal
        delay(200);
        tone(BUZZER_PIN,370,400); //Send 1KHz sound signal
        delay(200);
        noTone(BUZZER_PIN); //Stop sound
        //buzzer
        lcd.setCursor(0, 0);

        lcd.print(" You lose!");

        lcd.setCursor(0, 1);

        lcd.print("Length: ");

        lcd.setCursor(8, 1);

        lcd.print(snakeLength);

        break;

    case GAME_WIN:

        lcd.clear();

        lcd.setCursor(0, 0);

        lcd.print("You won. Congratz!");

        lcd.setCursor(0, 1);

```

```

lcd.print("Length: ");

lcd.setCursor(8, 1);

lcd.print(snakeLength);

break;

case GAME_PLAY:

for(size_t i=0; i<snakeLength; i++)

graphic_add_item(snakePosHistory[i].x, snakePosHistory[i].y, GRAPHIC_ITEM_A);

graphic_add_item(applePos.x, applePos.y, GRAPHIC_ITEM_B);

graphic_flush();

break;

case GAME_MENU:

//Do nothing

break;

}

}

void setup() {

lcd.init();

lcd.backlight();

lcd.print(" Snake Game");

// lcd.setCursor(0, 1);

// lcd.print("- Sadale.net");

pinMode(6, INPUT);

```

```

pinMode(7, INPUT);

pinMode(BUZZER_PIN, OUTPUT);

graphic_generate_characters();

int size = sizeof(durations) / sizeof(int);

//buzzer code
for (int note = 0; note < size; note++) {
    //to calculate the note duration, take one second divided by the note type.
    //e.g. quarter note = 1000 / 4, eighth note = 1000/8, etc.
    int duration = 1000 / durations[note];
    tone(BUZZER_PIN, melody[note], duration);

    //to distinguish the notes, set a minimum time between them.
    //the note's duration + 30% seems to work well:
    int pauseBetweenNotes = duration * 1.30;
    delay(pauseBetweenNotes);

    //stop the tone playing:
    noTone(BUZZER_PIN);

}
//buzzer code ends
lcd.setCursor(0, 2);
lcd.print("Press right button");

lcd.setCursor(11,3);
lcd.print("to start");

gameState = GAME_MENU;

}

void loop() {

    lcd.setCursor(0, 0);

    if(digitalRead(BUTTON_LEFT)==HIGH){

        if(debounce_activate_edge(&debounceCounterButtonLeft)&&!thisFrameControlUpdated){

```

```

switch(gameState){

case GAME_PLAY:

switch(snakeDirection){

case SNAKE_LEFT: snakeDirection=SNAKE_DOWN; break;

case SNAKE_UP: snakeDirection=SNAKE_LEFT; break;

case SNAKE_RIGHT: snakeDirection=SNAKE_UP; break;

case SNAKE_DOWN: snakeDirection=SNAKE_RIGHT; break;

}

thisFrameControlUpdated = true;

break;

case GAME_MENU:

game_init();

break;

}

}

}else{

debounce_deactivate(&debounceCounterButtonLeft);

}

lcd.setCursor(8, 0);

if(digitalRead(BUTTON_RIGHT)==HIGH){

if(debounce_activate_edge(&debounceCounterButtonRight)&&!thisFrameControlUpdated){

switch(gameState){

case GAME_PLAY:

```



```

switch(snakeDirection){

case SNAKE_LEFT: snakeDirection=SNAKE_UP; break;

case SNAKE_UP: snakeDirection=SNAKE_RIGHT; break;

case SNAKE_RIGHT: snakeDirection=SNAKE_DOWN; break;

case SNAKE_DOWN: snakeDirection=SNAKE_LEFT; break;

}

thisFrameControlUpdated = true;

break;

case GAME_MENU:

game_init();

break;

}

}

}else{

debounce_deactivate(&debounceCounterButtonRight);

}

if(millis()-lastGameUpdateTick>gameUpdateInterval){

game_calculate_logic();

game_calculate_display();

lastGameUpdateTick = millis();

thisFrameControlUpdated = false;

}

}

```

Explanation of Code

In these lines, constant integer variables `digitalRead(pin)`, `switch (expression)`, `game_init()`, `game_calculate_logic()`, `game_calculate_display()`, and `millis()` are declared and initialized.

These variables represent the digital pins to which the sensor, motor, and buzzer are connected, respectively.

Arduino.h: Standard Arduino library.

pitches.h: Header file containing pitch values for musical notes.

LiquidCrystalI2C.h: Library for interfacing with LCD displays using I2C communication.

Standard C libraries `stdlib.h` and `limits.h`

Arrays for storing melody and duration values for generating buzzer sounds. Arrays for representing graphical elements (blocks and apples). Variables for game state, snake position, apple position, etc.

The `setup()` function is executed only once when the Arduino board is powered up or reset. In the `setup()` function: Initializes the LCD display, buttons, and buzzer. Generates custom characters. Plays a startup melody. Displays a menu message.

In the `loop()` function: Reads button inputs. Updates game logic (snake movement, collision detection, etc.). Updates the display based on the game state. Plays buzzer sounds for specific events (e.g., winning or losing the game). The game initializes with a menu message on the LCD display. Players can start the game by pressing the right button.

During gameplay, players control the snake's direction using the left and right buttons. The snake moves across the grid, collecting apples and avoiding collisions with walls and its own body. If the snake collides with a wall or itself, the game ends and displays a "You lose" message. If the snake collects all available apples, the game ends and displays a "You won" message.

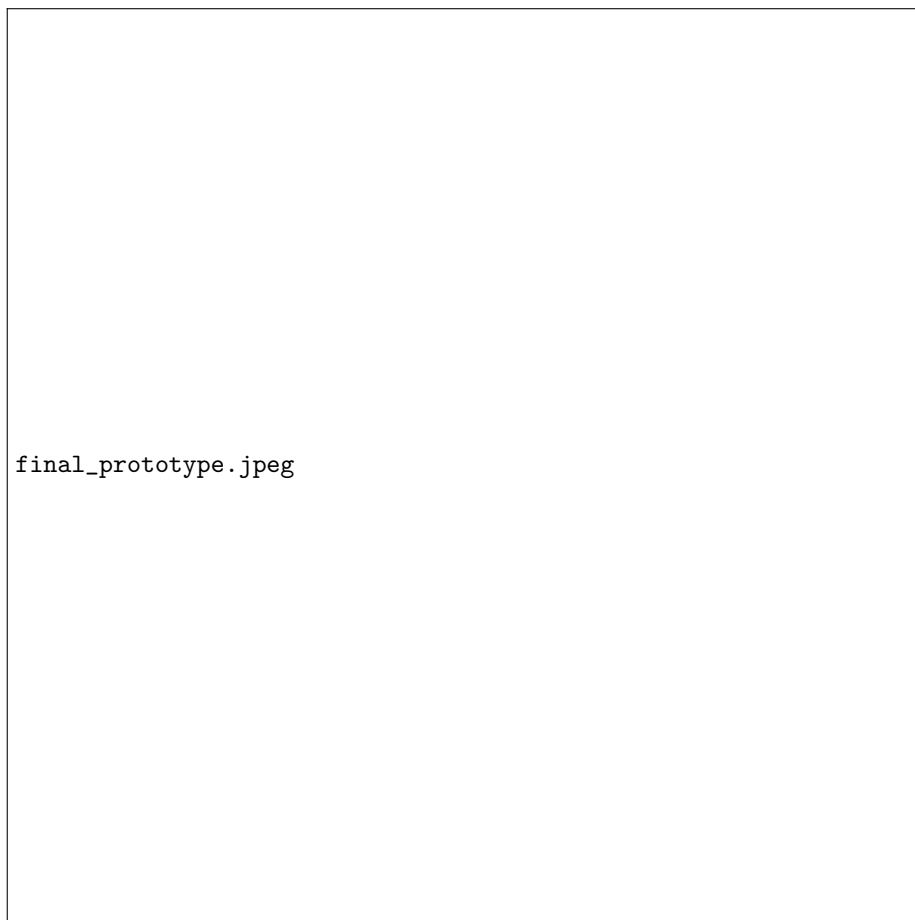


Figure 1: prototype cicuit

Result

The Retro Snake project successfully implemented core gameplay elements, faithfully recreating the experience of the original Snake game. Players can control the snake using push buttons, navigate through the game grid, and collect food items to grow longer. The game features intuitive controls, responsive gameplay, and immersive visual feedback, capturing the charm of the classic Snake experience while showcasing the capabilities of Arduino technology. The Retro Snake project has successfully recreated the classic Snake game experience using Arduino technology. Leveraging Arduino Uno board, push buttons for player input, and an LCD display for visualization, the project faithfully captures the essence of the original game. Players can control the snake, navigate through the game grid, and collect food items to grow longer. The game features responsive controls, intuitive gameplay mechanics, and immersive visual feedback, providing an engaging experience reminiscent of classic arcade games.

Conclusion

In conclusion, the Retro Snake project demonstrates the versatility and potential of Arduino in game development. By combining retro gameplay with modern technology, we have revived the nostalgia of the original Snake game and showcased the capabilities of Arduino hardware. This project highlights the enduring appeal of retro gaming and the accessibility of Arduino for creating fun and immersive gaming experiences. Moving forward, Arduino continues to offer exciting opportunities for innovation in game development, inspiring creators to explore new possibilities and push the boundaries of interactive entertainment.

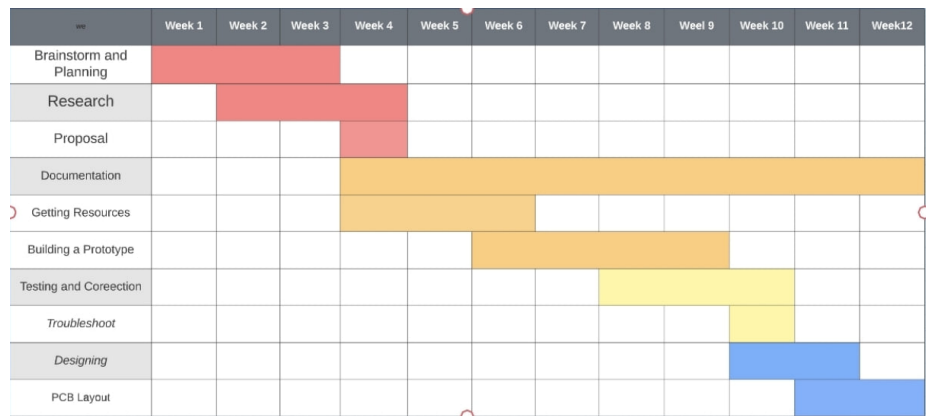


Figure 2: Gantt chart